The arguments to *runWriter* are matched to the *bind* function definition in the *Writer Monad*. Thus, x == 6, v == 'doubled 3', and f == 'double'. The function application of 'f x' is 'double 6' which yields '(12, "doubled 6")'. Thus y is 12 and v' is 'doubled 6'. The result is wrapped into a *Writer Monad* with y as 12, and the string v concatenated with v' to give 'doubled 3 doubled 6'. This example is useful as a logger, where you want a result and log messages appended together. As you can see, the output differs with input, and hence this is impure code that has side effects.

When you have data types, classes and instance definitions, you can organise them into a module that others can reuse. To enclose the definitions inside a module, prepend them with the module keyword. The module name must begin with a capital letter followed by a list of types and functions that are exported by the module. For example:

```
module Control.Monad.Writer.Class (
    MonadWriter(..),
    listens,
    censor,
  ) where

...
```

You can import a module in your code or at the GHCi prompt, using the following command:

```
import Control.Monad.Writer
```

If you want to use only selected functions, you can selectively import them using:

```
import Control.Monad.Writer(listens)
```

If you want to import everything except a particular function, you can hide it while importing, as follows:

```
import Control.Monad.Writer hiding (censor)
```

If two modules have the same function names, you can explicitly use the fully qualified name, as shown below:

```
import qualified Control.Monad.Writer
```

You can then explicitly use the 'listens' functions in the module using *Control.Monad.Writer.listens*. You can also create an alias using the 'as' keyword:

```
import qualified Control.Monad.Writer as W
```

You can then invoke the 'listens' function using *W.listens*.

Let us look at an example of the iso8601-time 0.1.2 Haskell package. The module definition is given below:

```
module Data.Time.ISO8601
  ( formatISO8601
  , formatISO8601Millis
  , formatISO8601Micros
  , formatISO8601Nanos
  , formatISO8601Picos
  , formatISO8601Javascript
  , parseISO8601
  ) where
```

It then imports a few other modules:

```
import Data.Time.Clock (UTCTime)
import Data.Time.Format (formatTime, parseTime)
import System.Locale (defaultTimeLocale)
import Control.Applicative ((<|>))
```

This is followed by the definition of functions. Some of them are shown below:

```
-- | Formats a time in ISO 8601, with up to 12 second decimals.
--
-- This is the `formatTime` format @%FT%T%Q@ == @%Y-%m-
%dT%H:%M:%S%Q@.
formatISO8601 :: UTCTime -> String
formatISO8601 t = formatTime defaultTimeLocale "%FT%T%QZ" t

-- | Pads an ISO 8601 date with trailing zeros, but lacking the
trailing Z.
--
-- This is needed because `formatTime` with "%Q" does not
create trailing zeros.
formatPadded :: UTCTime -> String
formatPadded t
  | length str == 19 = str ++ ".000000000000"
  | otherwise        = str ++ "000000000000"
  where
    str = formatTime defaultTimeLocale "%FT%T%Q" t

-- | Formats a time in ISO 8601 with up to millisecond
precision and trailing zeros.
-- The format is precisely:
-- >YYYY-MM-DDTHH:mm:ss.sssZ
formatISO8601Millis :: UTCTime -> String
formatISO8601Millis t = take 23 (formatPadded t) ++ "Z"
...
```

The availability of free and open source software allows you to learn a lot from reading the source code, and it is a very essential practice if you want to improve your programming skills. 

By: Shakthi Kannan

The author is a free software enthusiast and blogs at *shakthimaan.com*.